Climate Threat & Resistance Tracker API

COMP3011 Web Services and Web Data: Technical Report

Live endpoint: web-production-20ec6.up.railway.app • **Docs:** `/docs` • **Version:** 1.0.0

1 Project Overview

The Climate Threat and Resistance Tracker API is a fully deployed RESTful web service that maps the large-scale environmental threats such as oil pipelines, deforestation actors and polluters, along with their adversarial relationship with the civil society organisations opposing them. This area was chosen as it seemed both substantive and incredibly relevant within the current environmental crisis, both on a domestic and international scale. When faced with the scope of Modern-day environmental damage, specifically surrounding environmental externalities, it is common for individuals to feel powerless when moving against large scale corporate actors. Thus, the use of an API as an aid for uses to build a grounded understanding what organisations are fighting for environmental welfare seemed increasingly relevant. Additionally, the analytical richness of this domain allows the API to go well beyond simple CRUD and provide genuinely useful intelligence through composite analytics and a risk-scoring engine.

The system manages six relational entities (`Threat`, `Company`, `ThreatCompany`, `Organisation`, `Campaign`, `User`), exposes over 40 endpoints, and is deployed live on Railway with a managed PostgreSQL instance. The database is seeded with real-world cases including Rosebank Oilfield, the Willow Project, the East African Crude Oil Pipeline (EACOP), and JBS Amazon deforestation, making every analytical endpoint return substantively meaningful results rather than synthetic data.

2 Technology Stack Justification

2.1 Programming Language: Python 3.14

Python was chosen as the primary language as Python's async ecosystem has matured significantly, with `asyncio` as a first-class citizen in the standard library. The combination of Pydantic's type system with Python's type hints produces self-documenting, strictly-validated code. Python is also the dominant language in environmental data science tooling, which aligns naturally with the project.

The primary alternative considered was **Go with the Fiber framework**, which offers superior throughput. Go was rejected because its lack of mature ORM tooling and the absence of automatic OpenAPI generation would have required substantially more boilerplate to achieve equivalent documentation quality.

2.2 Framework: FastAPI

FastAPI was chosen over Django REST Framework (DRF) and Flask for three specific reasons:

- **Automatic OpenAPI generation:** FastAPI generates a fully interactive Swagger UI at `/docs` from Pydantic schemas alone, with no additional configuration. Documentation is always in sync with the implementation by construction.
- **Async-native architecture:** FastAPI is built on Starlette and supports `async`/`await` throughout. Combined with async SQLAlchemy and `asyncpg`, database calls are non-blocking so the API is better able to serve concurrent analytical queries.
- **Pydantic v2 integration:** Request validation, response serialisation, and schema generation are unified through a single library. There is no impedance mismatch between the validation layer and the documentation layer.

DRF was rejected because its synchronous ORM would have required a parallel async library. Flask was rejected as too minimal, lacking built-in schema validation and requiring significant additional configuration for production-grade output.

2.3 Database: PostgreSQL 18

The domain has an inherently complex relational structure with a single threat involving multiple companies in different roles as an organisation can run multiple campaigns against multiple threats. Junction tables and foreign key constraints enforce this integrity at the database level rather than in application code. SQLite was unsuitable for production deployment. MongoDB was considered for schema flexibility but rejected because the data model's relationships benefit from enforced referential integrity resulting in constraints that a document store would push into application logic. PostgreSQL's `ILIKE` operator also enabled case-insensitive full-text search without a separate search service, and `DateTime(timezone=True)` column typing correctly handles timezone-aware datetimes.

2.4 ORM and Migrations: SQLAlchemy 2.0 and Alembic

SQLAlchemy's 2.0 async API with the `asyncpg` driver provided a clean separation between the database and application layers. Alembic was used for versioned schema migrations: every schema change is committed as a discrete migration file, producing an auditable history. The deployment configuration runs `alembic upgrade head` as a pre-deploy command on Railway, ensuring migrations are applied atomically before traffic is served.

2.5 Deployment: Railway

Railway was selected over PythonAnywhere (the module-recommended option) because it provides managed PostgreSQL, automatic GitHub deployment, and environment variable injection in a single platform.

3 Architectural Design

3.1 Layered Service Pattern

The application is structured in three distinct layers. The **router layer** (`app/routers/`) handles HTTP concerns only: request parsing, response serialisation, status codes, and rate limit decorators. The **service layer** (`app/services/`) contains business logic decoupled from HTTP. The **ORM layer** (`app/models/`) defines the database schema, relationships, and constraints. This separation means the business logic and data models require no changes if the framework were replaced.

3.2 Schema Design

Every endpoint has explicit Pydantic request and response schemas. The `Create`, `Update`, and `Response` triad is implemented for all six entities. Analytics endpoints have dedicated response schemas (e.g. `CO2AtRiskResponse`, `ThreatRiskScoreResponse`) rather than returning untyped dictionaries, ensuring Swagger documents the full response structure. The `ErrorResponse` schema is referenced in every endpoint that can produce a 400, 401, 403, or 404.

3.3 Composite Risk Score

The `GET /analytics/threats/{id}/risk-score` endpoint synthesises four dimensions into a 0-100 score: CO2 impact (40 pts, normalised against a 500Mt upper bound), corporate exposure (25 pts, number of linked companies capped at 5), resistance gap (25 pts, reduced by active opposition campaigns), and geographic regulatory risk (10 pts, country-level multiplier). Bands are: LOW (0–24), MEDIUM (25–49), HIGH (50–74) and CRITICAL (75–100). The methodology is returned in the response body itself, making the scoring transparent without external

documentation.

4 Challenges, Testing, and Lessons Learned

4.1 Timezone-aware Datetimes

The most persistent technical challenge was the interaction between Python's datetime objects and PostgreSQL's timestamp types. The initial implementation used `datetime.utcnow()`, which produces timezone-naive datetimes. When columns were upgraded to `DateTime(timezone=True)`, asyncpg correctly rejected these, producing `DataError: can't subtract offset-naive and offset-aware datetimes`.

The fix required replacing every `datetime.utcnow()` call with `datetime.now(timezone.utc)`, and updating `onupdate` callbacks from `onupdate=datetime.utcnow` (a function reference) to `onupdate=lambda: datetime.now(timezone.utc)` (a deferred lambda). The lesson was precise: passing a function reference and deferring its call are semantically distinct in Python, and the distinction matters here because SQLAlchemy calls the function at insert time, not at class definition time.

4.2 Schema Migration Management

The early decision to use `Base.metadata.create_all()` during development created friction when adding the `land_area_affected_km2` column post-deployment: it required a manual `ALTER TABLE` because `create_all` does not modify existing tables. Migrating to Alembic earlier would have avoided this, teaching that schema migration tooling should be adopted at project inception, not retrofitted.

4.3 Testing Approach

A 44-test async suite was implemented using `pytest-asyncio` and `httpx`. A separate test database (`climate_test_db`) is isolated from the development instance. The test URL is derived by string replacement on `settings.DATABASE_URL`, so credentials are never duplicated. Session-scoped fixtures handle database setup and auth headers, avoiding repeated registration across tests. Tests cover happy paths, 401 rejections, 404 not-found cases, 422 validation failures, filtering logic, and the composite risk score endpoint. The suite passes consistently at 44/44 with 0 warnings.

5 Limitations and Future Development

5.1 Current Limitations

- **No Event model:** The planned `Event` entity for discrete real-world occurrences (court rulings, protests, spills) was not implemented, leaving the temporal dimension absent.
- **Static geographic risk table:** The risk score uses a hardcoded country dictionary. A production implementation would source indices from World Bank Governance Indicators.
- **No token refresh:** JWT tokens expire after 30 minutes with no refresh mechanism.
- **Inconsistent pagination:** `skip/limit` is implemented on `/threats` and `/companies` but not uniformly across all list endpoints.
- **No account lockout:** Repeated failed login attempts are rate-limited but not subject to lockout, leaving a residual brute-force exposure at sustained low request rates.

5.2 Future Development

- **Event model and timeline endpoint:** Tracking discrete events would enable temporal analysis across threats and campaigns.
- **External dataset integration:** Automated ingestion from Global Forest Watch, Carbon Tracker, or UK Land Registry.

- **MCP server exposure:** Wrapping the API as a Model Context Protocol server would allow LLM agents to query it directly, enabling natural-language environmental intelligence queries.
- **WebSocket subscriptions:** Real-time updates on threat status changes.
- **API versioning:** Prefixing routes with `/api/v1` and implementing a formal deprecation policy.

6 Generative AI Declaration and Usage Analysis

6.1 Declaration

Generative AI tools were used throughout this project. The primary tool was **Claude (Anthropic)** accessed via the `claude.ai` web interface. Two distinct sessions were conducted. Examples of exported conversation logs are attached as supplementary material per the module requirements.

6.2 Session 1: Primary Development Assistant

The first session was a continuous, conversation spanning the entire development lifecycle from architecture planning to deployment. Claude was used at several levels of abstraction:

- **Architecture planning:** Claude was asked to weigh a range of comprehensive API architectures using the project brief insight, this lead to the selection of a six-model schema, endpoint taxonomy, and layered service pattern. This was design reasoning, not code generation, and the output was critically evaluated before adoption.
- **Debugging:** Claude diagnosed non-trivial errors including the timezone-naive/aware conflict and the `ThreatCompany` relationship resolution failure at startup. In both cases, the diagnosis was correct and the fix was understood before application.
- **Boilerplate acceleration:** Repetitive patterns such as CRUD routers, Pydantic schemas, and test fixtures were generated by Claude and adapted. This is analogous to a code snippet library at a higher level of abstraction.
- **Documentation generation:** Claude aided in creating response schemas for analytics endpoints and the structured API documentation PDF through iterative feedback loops. Also helping with proofreading, phrasing chooses and latex formating within this report.

6.3 Session 2: Adversarial Examination

The second session took a deliberately unconventional approach. Claude was prompted to *"berate the API like an examiner that personally hated it but had to stay within the limits of the marking outline."* This was not a reflection of the actual marking process but a creative technique for surfacing blind spots that a supportive development session tends to miss.

The output was substantively useful: it identified missing response schemas on analytics endpoints, the absence of `UserUpdate`, inconsistent pagination, and undocumented error responses, all of which were subsequently addressed. This approach treats the model as an adversarial reviewer rather than a collaborative assistant, producing qualitatively more critical feedback.

A subsequent prompt asked Claude to *"play a role playing exercise to improve my api security, if you wanted comprehensively break down how to hack, sabotage, and hinder the API how would safeguard against that. you don't need to know tell me how to facilitating attacks just how to prevent you efforts"* after sharing the repository. This functioned as a structured security audit. Claude identified brute-force login as a vector (mitigated by rate limiting), JWT secret exposure (mitigated by `pydantic-settings` and `.env` gitignore exclusion), and SQL injection (mitigated by SQLAlchemy's parameterised queries). The account lockout gap, identified in this session, remains a current limitation.

6.4 Reflective Analysis

The most valuable use of GenAI in this project was **architectural reasoning and adversarial review**, not code generation. The model's capacity to reason about design trade-offs and to simulate an examiner's or attacker's perspective produced insights that would have required substantially more time through solo development.

The principal risk in this mode of working is uncritical acceptance of generated output. This was managed by ensuring every fix was understood before application, and by maintaining commit discipline: each commit message reflects genuine comprehension of the change rather than copying generated text verbatim.

7 Summary

Component	Implementation
Framework	FastAPI with async SQLAlchemy 2.0
Database	PostgreSQL 18 (Railway-managed)
Migrations	Alembic with versioned history
Authentication	JWT (python-jose), bcrypt passwords, role-based access
Rate limiting	slowapi (100/min read, 20/min write, 60/min analytics)
Testing	pytest-asyncio + httpx, 44 tests, 0 warnings
Deployment	Railway (auto-deploy from GitHub, pre-deploy migrations)
Documentation	Swagger UI (auto), ReDoc, PDF export
Endpoints	40+ including 11 analytics and composite risk score
Seed data	6 real-world threats, 5 companies, 6 organisations

The project demonstrates that a well-designed, domain-driven API can significantly exceed minimum requirements while remaining architecturally coherent. The adversarial use of AI for review rather than purely for generation. This was the most significant methodological innovation, and one that produced measurable improvements to the final submission.